

Introduction to Machine Learning

Misha Tan

ICEF

House price prediction

Suppose we observe the characteristics and sale prices of several houses.

House	Area (m ²)	Rooms	Age	Price (\$000)
A	55	1	12	140
B	78	2	6	230
C	105	3	20	315
New	82	2	8	?

Question

Can the observed houses help us estimate the price of the new house?

Objects and features

An **object** is an abstract entity that we analyse. A house, person, firm, or transaction can be an object.

We represent object i by recorded characteristics called **features**:

$$x_i = (x_{i1}, \dots, x_{id})$$

is its **object vector**. Its coordinate x_{ij} is feature j , and d is the number of features. From now on, x_i denotes the object through this representation.

Feature type	House-price example
Numerical	area, number of rooms, age
Categorical	district, building type
Binary	lift present: yes or no
Ordinal	condition: poor, average, good

Training data and prediction model

The **answer**, or **target**, y_i is the value associated with object x_i that we want to predict. A **training example** is an object–target pair (x_i, y_i) .

The training sample is

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{\ell},$$

where ℓ is the number of observed objects.

A **prediction model** maps an object vector to a predicted answer:

$$x \longmapsto \hat{y} = f(x).$$

The hat distinguishes the predicted answer $\hat{y}$ from the observed answer y .

Learning a prediction model

A learning algorithm uses the training sample to choose a fitted model $\hat{f}$:

$$\mathcal{D} \longmapsto \hat{f}, \quad \hat{y} = \hat{f}(x).$$

Learning goal

Find a model that predicts answers accurately for objects not used during training.

A loss function will define prediction quality. A train–test split or cross-validation will later measure performance on unseen objects.

Machine learning

Working definition

Machine learning is a field of study that examines methods for extracting patterns from available data and using those patterns to solve tasks on new data.

Types of learning problems

Supervised learning

The data contain object–target pairs:

$$(x_i, y_i).$$

- ▶ Regression: numerical target
- ▶ Classification: categorical target

House price prediction is a supervised regression problem.

Unsupervised learning

The data contain objects but no targets:

$$x_i.$$

- ▶ Clustering
- ▶ Density estimation

Linear regression

Machine learning uses many different models. One of the simplest and most widely used is **linear regression**: it predicts by a weighted linear combination of numerical features.

$$f(x) = w_0 + \sum_{j=1}^d w_j x_j, \quad \hat{y}_i = f(x_i).$$

- ▶ The adjustable parameters $w_0, \dots, w_d$ are called **weights**, or coefficients. The weight w_0 is the intercept.
- ▶ Holding the other features fixed, increasing x_{ij} by one changes the prediction by w_j .
- ▶ The model is linear in the parameters w , while the features may include transformations of the original variables.

Vector form for one observation

To include the intercept in the vector notation, add a constant feature $x_{i0} = 1$.
From now on, write

$$x_i = (1, x_{i1}, \dots, x_{id})^T \in \mathbb{R}^{d+1}, \quad w = (w_0, w_1, \dots, w_d)^T \in \mathbb{R}^{d+1}.$$

Then the prediction for observation i is

$$\hat{y}_i = f(x_i) = x_i^T w = w_0 + \sum_{j=1}^d w_j x_{ij}.$$

The superscript T denotes transpose. For column vectors $a, b \in \mathbb{R}^{d+1}$, their inner product is

$$a^T b = \sum_{j=0}^d a_j b_j.$$

Matrix form for the full sample

Stack the object vectors and targets:

$$X = \begin{bmatrix} x_1^T \\ \vdots \\ x_\ell^T \end{bmatrix} \in \mathbb{R}^{\ell \times (d+1)}, \quad y = \begin{bmatrix} y_1 \\ \vdots \\ y_\ell \end{bmatrix} \in \mathbb{R}^\ell.$$

Matrix multiplication gives all predictions at once:

$$\hat{y} = Xw = \begin{bmatrix} x_1^T w \\ \vdots \\ x_\ell^T w \end{bmatrix} \in \mathbb{R}^\ell.$$

Loss functions

To compare forecasts, we need a numerical measure of prediction quality.

A **loss function**

$$L : \mathbb{R} \times \mathbb{R} \rightarrow [0, \infty)$$

assigns a nonnegative cost $L(y, \hat{y})$ to a target y and forecast $\hat{y}$.

For observation i ,

$$\hat{y}_i = f(x_i), \quad \text{forecast loss} = L(y_i, \hat{y}_i).$$

Smaller loss means a more accurate forecast.

Common regression losses

Loss	Definition
Squared loss	$L_{\text{sq}}(y, \hat{y}) = (y - \hat{y})^2$
Absolute loss	$L_{\text{abs}}(y, \hat{y}) = y - \hat{y} $

- ▶ Squared loss gives large prediction errors greater influence and is differentiable everywhere.
- ▶ Absolute loss is less sensitive to large prediction errors but is not differentiable at $y = \hat{y}$.

The objective function

The **objective function** is the average loss on the training sample:

$$Q(w) = \frac{1}{\ell} \sum_{i=1}^{\ell} L(y_i, f(x_i)).$$

Training means solving the minimization problem

$$\boxed{\min_{w \in \mathbb{R}^{d+1}} Q(w)}.$$

A parameter vector that solves this problem is denoted by $\hat{w}$.

Mean squared error and OLS

With squared loss, the objective is the **mean squared error**:

$$Q(w) = \text{MSE}(w) = \frac{1}{\ell} \sum_{i=1}^{\ell} (f(x_i) - y_i)^2.$$

Ordinary least squares

OLS chooses the parameters of a linear model by minimizing the sum, or equivalently the mean, of squared prediction errors.

Multiplying an objective by a positive constant does not change its minimizers.

Least squares in matrix form

For $z = (z_1, \dots, z_\ell)^\top$, the Euclidean norm is

$$\|z\|_2 = \sqrt{z_1^2 + \dots + z_\ell^2}, \quad \|z\|_2^2 = z^\top z.$$

The OLS objective is therefore

$$Q(w) = \frac{1}{\ell} \|Xw - y\|_2^2 = \frac{1}{\ell} (Xw - y)^\top (Xw - y).$$

The vector $Xw - y$ contains the differences between the predictions and observed targets.

Expanding the OLS objective

Starting from

$$Q(w) = \frac{1}{\ell} (Xw - y)^T (Xw - y),$$

expansion gives

$$Q(w) = \frac{1}{\ell} (w^T X^T X w - 2w^T X^T y + y^T y).$$

The cross terms combine because

$$y^T X w = (y^T X w)^T = w^T X^T y.$$

The last term is constant with respect to w .

Normal equations and the OLS solution

Since $X^T X$ is symmetric, for $k = 0, \dots, d$,

$$\frac{\partial Q(w)}{\partial w_k} = \frac{2}{\ell} \left[(X^T X w)_k - (X^T y)_k \right].$$

At a differentiable local minimum, every coordinate derivative equals zero:

$$\frac{\partial Q(\hat{w})}{\partial w_k} = 0 \quad \text{for all } k.$$

Setting these derivatives to zero gives the **normal equations**

$$X^T X \hat{w} = X^T y.$$

For a linear model with MSE, Q is differentiable and convex, so the first-order conditions are sufficient for a global minimum. If $X^T X$ is invertible,

$$\hat{w} = (X^T X)^{-1} X^T y.$$

Geometric interpretation: numerical example

Take $\ell = 3$ observations and $d = 2$ features. For this illustration, omit the intercept:

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad y = \begin{bmatrix} 2 \\ 1 \\ 4 \end{bmatrix}.$$

The columns $X_1 = (1, 0, 1)^T$ and $X_2 = (0, 1, 1)^T$ belong to $\mathbb{R}^3$. Every possible fitted vector is

$$Xw = w_1X_1 + w_2X_2 = (w_1, w_2, w_1 + w_2)^T.$$

Hence the column space is the plane

$$\text{col}(X) = \left\{ (a, b, a + b)^T : a, b \in \mathbb{R} \right\}.$$

Since $4 \neq 2 + 1$, the vector y does not lie in this plane, so an exact fit is impossible.

OLS as a projection

OLS chooses $\hat{y} = X\hat{w} \in \text{col}(X)$ closest to y . The residual $r = y - \hat{y}$ is perpendicular to the plane, which is expressed by $X^T r = 0$.

Limitations of closed-form OLS

The formula

$$\hat{w} = (X^T X)^{-1} X^T y$$

is useful, but it does not solve every training problem.

- ▶ $X^T X$ may be singular, so its inverse does not exist.
- ▶ With many observations, forming $X^T X$ is costly; with many features, solving the resulting system is also costly.
- ▶ Other loss functions or model families may have no analytical solution.

These problems motivate regularization and iterative optimization methods.

The gradient

Let $Q : \mathbb{R}^m \rightarrow \mathbb{R}$ be differentiable, where m is the number of parameters. Its **gradient** at w is the vector

$$\nabla Q(w) = \left(\frac{\partial Q(w)}{\partial w_1}, \dots, \frac{\partial Q(w)}{\partial w_m} \right)^T.$$

Each component measures how Q changes when one parameter changes while the others are held fixed.

- ▶ $\nabla Q(w)$ points in the direction of the fastest local increase of Q .
- ▶ $-\nabla Q(w)$ points in the direction of the fastest local decrease.
- ▶ $\|\nabla Q(w)\|_2$ describes how steep the objective is near w .

Gradient descent

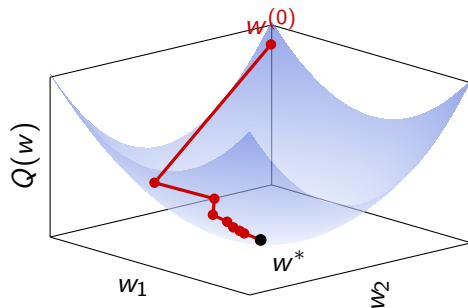
Choose an initial vector $w^{(0)}$. At iteration t ,

$$w^{(t+1)} = w^{(t)} - \eta_t \nabla Q(w^{(t)})$$

where $\eta_t > 0$ is the **learning rate**, or step size.

Step choice. A very small η_t gives slow progress. A very large η_t may overshoot or diverge.

Stop when the gradient is small, the objective or parameters barely change, or the iteration limit is reached.



The red points are GD iterates on $Q(w_1, w_2) = \frac{1}{2}(w_1^2 + 6w_2^2)$ with $\eta_t = 0.22$.

Limitations of gradient descent

1. **Cost of one iteration.** If

$$Q(w) = \frac{1}{\ell} \sum_{i=1}^{\ell} L_i(w), \quad \nabla Q(w) = \frac{1}{\ell} \sum_{i=1}^{\ell} \nabla L_i(w),$$

full GD evaluates ℓ individual gradients at every iteration.

2. **Non-convex objectives.** GD may approach a local minimum or a saddle point instead of a global minimum. **Multistart** runs GD from several initial values and keeps the result with the smallest objective value, but it gives no guarantee.
3. **Convergence speed.** Under suitable conditions for a convex differentiable objective, there exists $C > 0$ such that, for all sufficiently large k ,

$$0 \leq Q(w^{(k)}) - Q(w^*) \leq \frac{C}{k}.$$

Therefore, the objective gap converges at rate $O(1/k)$.

Stochastic gradient descent

Let $L_i(w) = L(y_i, f(x_i))$ be the loss on observation i . Then

$$Q(w) = \frac{1}{\ell} \sum_{i=1}^{\ell} L_i(w).$$

At iteration t , SGD samples an index i_t and uses one observation:

$$\boxed{w^{(t+1)} = w^{(t)} - \eta_t \nabla L_{i_t}(w^{(t)})}.$$

- ▶ One SGD step is cheap because it does not sum over all ℓ observations.
- ▶ Far from the minimum, an individual negative gradient often points roughly in the same direction as the full GD step.
- ▶ The direction is noisy, so a single step may increase the full objective even though the method improves it on average.

SGD convergence and mini-batches

Near a minimum, individual gradients can point in different directions.
For convex differentiable objectives, classical learning-rate conditions are

$$\sum_{t=0}^{\infty} \eta_t = \infty, \quad \sum_{t=0}^{\infty} \eta_t^2 < \infty.$$

For example, $\eta_t = c/(t+1)$ with $c > 0$ satisfies both conditions.
Under standard assumptions, the iterate $w^{(k)}$ can satisfy

$$\mathbb{E}\left[Q(w^{(k)}) - Q(w^*)\right] \leq \frac{c}{\sqrt{k}} = O\left(\frac{1}{\sqrt{k}}\right).$$

A **mini-batch** B_t of size b uses

$$w^{(t+1)} = w^{(t)} - \frac{\eta_t}{b} \sum_{i \in B_t} \nabla L_i(w^{(t)}).$$

Larger b reduces noise but costs more. The cases $b = 1$ and $b = \ell$ give SGD and full GD.

Momentum

Momentum keeps a running update vector $h^{(t)}$. Starting from $h^{(0)} = 0$,

$$h^{(t+1)} = \alpha h^{(t)} + \eta_t \nabla Q(w^{(t)}), \quad w^{(t+1)} = w^{(t)} - h^{(t+1)}.$$

Here $\eta_t > 0$ is the learning rate and $0 \leq \alpha < 1$ is the **momentum coefficient**. The learning rate is already included in $h^{(t+1)}$.

Repeated substitution gives

$$h^{(t+1)} = \eta_t \nabla Q(w^{(t)}) + \alpha \eta_{t-1} \nabla Q(w^{(t-1)}) + \dots$$

recent gradients receive more weight than older gradients.

- Directions that remain consistent accumulate, accelerating progress.
- Directions that alternate partly cancel, reducing zig-zagging.

Nesterov momentum

Ordinary momentum evaluates the gradient at the current point $w^{(t)}$. Nesterov momentum first looks in the direction in which momentum is about to move:

$$h^{(t+1)} = \alpha h^{(t)} + \eta_t \nabla Q(w^{(t)} - \alpha h^{(t)}), \quad w^{(t+1)} = w^{(t)} - h^{(t+1)}.$$

The point $w^{(t)} - \alpha h^{(t)}$ is the **look-ahead point**. Its gradient can correct the anticipated momentum step before the full update is made.

For stochastic optimization, ∇Q may be replaced by a mini-batch gradient evaluated at the look-ahead point.

AdaGrad

A global schedule decreases η_t at every SGD step. For a sparse feature, $\partial L_{i_t}(w^{(t)})/\partial w_j = 0$ on many steps. Then w_j does not move, but η_t still becomes smaller.

AdaGrad lets each coordinate decrease its step according to its own gradient history. Set $s_j^{(0)} = 0$ and, for a base learning rate $\eta > 0$, update

$$s_j^{(t+1)} = s_j^{(t)} + \left(\frac{\partial L_{i_t}(w^{(t)})}{\partial w_j} \right)^2,$$
$$w_j^{(t+1)} = w_j^{(t)} - \frac{\eta}{\sqrt{s_j^{(t+1)} + \epsilon}} \frac{\partial L_{i_t}(w^{(t)})}{\partial w_j}.$$

If the derivative is zero, neither w_j nor s_j changes. The coordinate-specific step shrinks only when coordinate j receives gradients.

Limitation: $s_j^{(t)}$ never decreases, so the steps may eventually become too small.

RMSProp

RMSProp remembers recent squared derivatives instead of accumulating all of them:

$$s_j^{(t+1)} = \rho s_j^{(t)} + (1 - \rho) \left(\frac{\partial L_{i_t}(w^{(t)})}{\partial w_j} \right)^2,$$

$$w_j^{(t+1)} = w_j^{(t)} - \frac{\eta}{\sqrt{s_j^{(t+1)}} + \varepsilon} \frac{\partial L_{i_t}(w^{(t)})}{\partial w_j}.$$

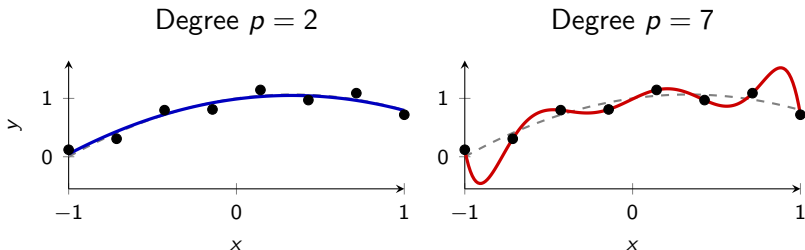
- ▶ $0 \leq \rho < 1$ controls how quickly old squared derivatives are forgotten.
- ▶ Large recent derivatives reduce the corresponding step; when they become smaller, the step can grow again.
- ▶ $\varepsilon > 0$ prevents division by zero.

Adam combines momentum with AdaGrad's coordinate-specific scaling, using the RMSProp version with exponential forgetting.

Polynomial regression and overfitting

A polynomial regression model of degree p has the form

$$f_p(x, w) = \sum_{j=0}^p w_j x^j.$$



Dots: training observations. Dashed curve: the relationship used to generate the data.

Overfitting

Overfitting occurs when a model performs much better on the training sample than on unseen data.

Training, validation, and test sets

Split the observations into three parts so that no observation appears in more than one set.

Data	Role
Training set	Fit the model parameters w .
Validation set	Compare models and choose hyperparameters.
Test set	Estimate final performance on unseen data after all choices have been made.

Hyperparameter

A hyperparameter is a modelling choice made outside parameter fitting. The polynomial degree p is one example.

K -fold cross-validation

Reserve the test set first. Split the remaining data into K subsets called **folds** and train K models.

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Round 1	Validate	Train	Train	Train	Train
Round 2	Train	Validate	Train	Train	Train
Round 3	Train	Train	Validate	Train	Train
Round 4	Train	Train	Train	Validate	Train
Round 5	Train	Train	Train	Train	Validate

Each observation is used for validation once and for training $K - 1$ times. The example above has $K = 5$.

Cross-validation for model selection

Let $Q_k(p)$ be the validation loss in fold k for a polynomial of degree p , after fitting the model on the other $K - 1$ folds. Then

$$CV(p) = \frac{1}{K} \sum_{k=1}^K Q_k(p), \quad \hat{p} \in \arg \min_{p \in \mathcal{P}} CV(p),$$

where $\mathcal{P}$ is the set of candidate degrees.

After selecting $\hat{p}$, there are two options:

1. Fit a final degree- $\hat{p}$ model using all observations included in cross-validation.
2. If this extra fit is too costly, average the predictions of the K existing models:

$$\hat{y}_{\text{avg}}(x) = \frac{1}{K} \sum_{k=1}^K a_k(x),$$

where $a_k(x)$ is the prediction from the model trained without fold k .

If a separate test set was reserved before cross-validation, evaluate the final predictor once on it.

Large coefficients and unstable fits

In the polynomial example, the degree-7 fit contains

$$f_7(x) = 0.99 + 1.38x - 0.57x^2 - 10.62x^3 + 26.07x^5 - 16.52x^7.$$

Let $w^{(p)} = (w_1, \dots, w_p)^T$ contain the non-intercept coefficients of the degree- p model. Then

$$\|w^{(2)}\|_2 \approx 0.68, \quad \|w^{(7)}\|_2 \approx 32.67.$$

- ▶ In the house-price example, two otherwise similar houses with almost the same area should have similar prices.
- ▶ A model with very large coefficients can turn a small change in area into an implausibly large change in the predicted price.

Regularization

A **regularizer** $R(w)$ measures an aspect of the parameter vector that we want to discourage. Regularized fitting solves

$$\hat{w}_\lambda \in \arg \min_{w \in \mathbb{R}^{d+1}} \{Q(w) + \lambda R(w)\}.$$

- ▶ $Q(w)$ is the training objective.
- ▶ $\lambda \geq 0$ is the **regularization strength**, chosen using validation or cross-validation.
- ▶ Larger λ gives more importance to the penalty and usually produces smaller coefficients.

Constrained view

For a corresponding coefficient budget $t \geq 0$, the same solution can be viewed as

$$\min_w Q(w) \quad \text{subject to} \quad R(w) \leq t.$$

Feature scaling before regularization

Important

Coefficient size depends on feature units. The same area measured in m^2 or km^2 gives different coefficients but identical predictions. Because L_1 and L_2 penalize coefficient size, we scale features first so that the comparison is fair.

For each numerical feature j , compute its mean $\bar{x}_j$ and standard deviation $s_j > 0$ on the training sample, then use

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}.$$

- ▶ The transformed feature has mean zero and standard deviation one.
- ▶ Use the same training values $\bar{x}_j$ and s_j for validation and test observations.
- ▶ The intercept is not scaled or penalized.

Ridge and lasso

The intercept w_0 is usually not penalized. For the remaining coefficients,

Method	Regularizer	Typical effect
Ridge (L_2)	$R_2(w) = \sum_{j=1}^d w_j^2 = \ w_{1:d}\ _2^2$	Shrinks coefficients towards zero; they usually remain nonzero.
Lasso (L_1)	$R_1(w) = \sum_{j=1}^d w_j = \ w_{1:d}\ _1$	May set some coefficients exactly to zero.

Sparse means that few coefficients are nonzero.

For ridge, let $P = \text{diag}(0, 1, \dots, 1)$. With a constant intercept column and $\lambda > 0$,

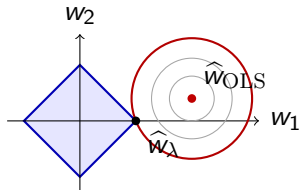
$$X^T X + \ell \lambda P \text{ is invertible, } \quad \hat{w}_{\text{ridge}} = (X^T X + \ell \lambda P)^{-1} X^T y.$$

This gives a unique solution even when $X^T X$ is singular.

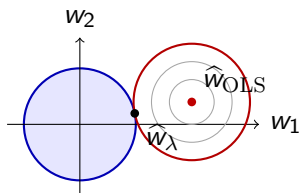
Why L_1 can produce exact zeros

With two coefficients, compare the feasible regions $R(w) \leq t$.

$$L_1: |w_1| + |w_2| \leq t$$



$$L_2: w_1^2 + w_2^2 \leq t^2$$



A corner lies on an axis, so here $\hat{w}_{\lambda,2} = 0$. The smooth boundary usually gives two nonzero coefficients.

The smallest loss level curve that reaches the blue feasible region determines the constrained solution.